

Course	CIP A106 Malware Analysis
Name	Janet Oluwatosin Michael
Laboratory	Lab 4 Lokibot Trojan Decompilation and Controlled Debug Analysis
Student Id	2026/EHIT/17073
Instructor	Mr Aminu Idris
Submission Date	08/10/2026

1. Introduction

The aim of this lab was to analyse Sample 3, which was provided as a LokiBot sample, and understand its possible behaviour using different malware analysis tools. I used both static analysis and debugger observations to examine the sample and identify activities that could be suspicious.

I used Exe2Aut to recover readable AutoIt-like code and Ghidra to examine the program's strings, imported functions, and entry point. I also used x32dbg and System Informer to observe the sample process and collect information about how it was started.

During the analysis, I found obfuscated code containing functions related to downloading files, launching files, writing data, and checking for certain processes. These findings helped me understand what the sample might be capable of doing. However, I did not treat the presence of these functions as proof that the activities were successfully carried out.

This report presents the tools used, the findings from the analysis, the indicators identified, and the evidence collected. It also explains the limitations of the analysis and distinguishes between confirmed observations and possible malware behaviour.

2. Environment and Tool Methodology

I carried out the analysis in a Windows virtual machine running Windows 10 Pro, version 2009, with OS build number 26200. The system had two processors and an AMD64 architecture.

I used Exe2Aut to recover readable AutoIt-like code from the sample. I used Ghidra 12.1.4 to examine the executable's entry point, strings, imported functions, and other program details. I also used x32dbg to inspect the sample in a debugger and System Informer to monitor the process list. PowerShell was used to confirm the process ID, executable path, parent process, and command line.

The sample was stored in the Sample3_Lokibot_Evidence folder. I intended to keep the virtual machine isolated during the analysis to reduce the risk of unwanted external communication. Evidence of the actual network setting will be included in the appendix.

I used the tools to examine the sample without assuming that every function found in the code had been executed. I recorded the findings based on the output available to me and treated possible malware activities as unconfirmed unless there was supporting evidence.

3. Sample Identification and Hashes

The sample was provided in an archive named Sample 3.zip. After extracting the files, I located the sample named sample.bin in the SampleContents\malware-sample-3 folder inside my evidence directory.

The original archive had a file size of 779,839 bytes, while the inner ZIP archive had a file size of 779,368 bytes. The extracted sample.bin file had a size of 1,217,024 bytes.

I calculated the SHA-256 hash of the original sample to identify the exact file used during the analysis. The recorded SHA-256 hash was:

0DA91175E7D72A7FF2BCB3FD93F2BA7BBE4045F9C4DEE5C9685C7FDF6DA622A6

The hash provides a way to check whether the file used for analysis matches the recorded sample. Although the lab identifies the sample as LokiBot, the filename and hash alone do not confirm its malware family. The classification must be considered alongside the analysis findings.

4. Exe2Aut Unpacking and Decompilation Findings

I used Exe2Aut to examine the sample and recover readable AutoIt-like code. The output contained many obfuscated variable names and hexadecimal strings, which made the code difficult to understand at first. However, some of the recovered code helped me identify functions that could be related to suspicious activities.

One finding was the use of InetGet() together with ShellExecute(). The code appeared to download a file to the temporary directory and then attempt to launch it. This could allow the sample to download and execute another file. However, I did not confirm that the download was successful or identify a confirmed external download URL.

I also found code that used resource-related functions, including FindResourceW, SizeofResource, LoadResource, and LockResource. Other recovered code appeared to write resource data to a file. This suggests that the sample may be capable of extracting data from its own resources, although the actual result of this activity was not confirmed.

Another suspicious finding was code that constructed a path using @StartupDir and a filename ending in .url. This could be related to persistence by creating a file in a startup location. However, I did not confirm that the file was successfully created or that persistence was established.

The code also contained checks for processes such as `vmtoolsd.exe` and `vbox.exe`. These checks may indicate that the sample was checking for a virtual machine environment. It also contained code that could attempt to close the running script process. These observations may suggest virtual-machine detection or an attempt to avoid analysis, but I could not confirm the exact purpose of these checks.

I also observed code that appeared to use a command prompt to delete the running script after a short delay. This could be an attempt to remove the sample file after execution. However, the code alone does not prove that the deletion occurred.

Overall, Exe2Aut helped me understand parts of the sample that were difficult to read in their original form.

5. Ghidra Static Analysis

I used Ghidra version 12.1.4 to examine the sample's executable structure, strings, imported functions, and entry point. I imported `sample.bin` as a 32-bit x86 Portable Executable (PE) file.

During the analysis, I found strings such as `AutoIt v3`, `/AutoIt3ExecuteScript`, `/AutoIt3ExecuteLine`, and `/AutoIt3OutputDebug`. These strings suggest that the sample is associated with the AutoIt scripting environment. I also found strings relating to file operations, process handling, networking, and registry operations. However, some of these strings may belong to the AutoIt runtime, so their presence does not prove that the sample performed every related activity.

The imported libraries included `KERNEL32.dll`, `ADVAPI32.dll`, `SHELL32.dll`, `USER32.dll`, `USERENV.dll`, `VERSION.dll`, `OLE32.dll`, and `WININET.dll`. The sample also had WinINET imports such as `InternetOpenW`, `InternetOpenUrlW`, `HttpSendRequestW`, `HttpQueryInfoW`, and `InternetCloseHandle`. These functions can support internet and HTTP communication, but the imports alone do not confirm that a connection was made or identify a command-and-control server.

I examined the entry point shown in Ghidra. At address `00427DCD`, the code called `__security_init_cookie`, followed by a jump at `00427DD2` to `FUN_00427C56`. This showed the initial control flow identified during my inspection. I did not treat these instructions alone as proof of the sample's full execution behaviour.

Ghidra displayed a warning that it failed to define some string data. This limited part of the analysis, so I considered the findings alongside the recovered code from Exe2Aut instead of relying on Ghidra alone.

Overall, Ghidra helped me identify the file as a 32-bit PE executable and examine its AutoIt-related strings, imported functions, and entry-point instructions.

6. Debugger Analysis Using x32dbg

I used x32dbg to inspect the sample during dynamic analysis. I loaded `sample.bin` into the debugger and examined the instructions displayed around the program's entry point.

At address 00B37DCD, the debugger displayed a CALL instruction that transferred control to another location in the sample. The next instruction, at 00B37DD2, was a JMP instruction. These instructions helped me observe the program's initial control flow.

I also inspected the debugger's view of Windows API functions, including VirtualAlloc. This function is used by Windows programs to allocate memory. However, the view I inspected showed the Windows API implementation rather than a confirmed call made by the sample. Therefore, I could not use this observation alone to conclude that the sample allocated memory for code injection or another malicious purpose.

The debugger also displayed INT3 instructions after the entry-point instructions. These are breakpoint instructions that can be used for debugging or padding. Their presence alone was not enough to confirm anti-debugging behaviour.

I used x32dbg alongside the static analysis findings to understand the sample's execution flow. Although the debugger provided useful information about the instructions and API functions, I did not confirm a specific malicious action from these observations alone. I therefore considered the debugger findings together with the process information collected using System Informer and PowerShell.

7. Process Monitoring Using System Informer and PowerShell

I used System Informer and PowerShell to inspect the sample's running process and collect information about how it was started.

Using PowerShell, I identified the process as sample.bin, with process ID (PID) 1052. The executable path was:

```
C:\Users\janet\Desktop\Sample3_Lokibot_Evidence\01_Original\SampleContents\malware-sample-3\sample.bin
```

The process information showed that its parent process ID was 5872, which belonged to x32dbg.exe. This was consistent with the sample being launched under the debugger during my analysis. The command line also showed the path to sample.bin.

System Informer was used to view the running processes and help monitor the sample while it was active. These observations confirmed that the sample process was running at the time the information was collected. The process monitoring results supported my investigation by confirming the sample's execution and providing information about its process ID, executable path, and parent process. I used these results together with the static analysis findings to build a more complete understanding of the sample.

8. Indicators of Compromise (IoCs)

During my analysis, I recorded the available indicators that could help identify the sample. These included the SHA-256 hash of the sample, its filename, and strings and functions observed during static analysis.

The SHA-256 hash of the extracted sample was:

0DA91175E7D72A7FF2BCB3FD93F2BA7BBE4045F9C4DEE5C9685C7FDF6DA622A6

The sample was named sample.bin and was located in the malware-sample-3 folder inside my evidence directory. The process ID observed during execution was 1052. This PID is useful for documenting the process during this analysis, but it is not a permanent indicator because process IDs can change between executions.

The recovered code contained references to InetGet(), ShellExecute(), resource-handling functions, and a possible startup .url file path. These may help guide further investigation, but they are behavioural clues rather than confirmed indicators of compromise.

I also found the string 127.0.0.1 during the review. This is a loopback address and does not, by itself, identify an external command-and-control server. I did not confirm an external IP address, domain name, or URL from the evidence reviewed.

The indicators collected during this investigation are summarised below.

Indicator	Value	Status
Sample filename	sample.bin	Confirmed
SHA-256	0DA91175E7D72A7FF2BCB3FD93F2BA7BBE4045F9C4DEE5C9685C7FDF6DA622A6	Confirmed
Process name	sample.bin	Observed during analysis
Process ID	1052	Observed; temporary
AutoIt-related strings	AutoIt v3, /AutoIt3ExecuteScript	Found in static analysis
External IP address	Not identified	Unconfirmed
External domain or URL	Not identified	Unconfirmed

The hash is the strongest identification indicator recorded for the exact sample file. Further

network and system monitoring would be needed to establish additional indicators, such as external connections, dropped files, registry changes, or persistence mechanisms.

9. MITRE ATT&CK Mapping

During my analysis, I reviewed the sample's recovered code, imported functions, strings, and debugger observations to identify possible malicious behaviours. However, I could not confidently confirm specific MITRE ATT&CK techniques from the evidence collected.

Some code suggested possible file downloading, file execution, and startup-file creation, but I did not confirm that these activities were successfully performed. I therefore decided not to assign MITRE ATT&CK technique IDs without sufficient evidence.

Further analysis, including confirmed execution behaviour and supporting system or network evidence, would be required before mapping the sample to specific MITRE ATT&CK techniques.

10. Analysis Limitations and Challenges

During the analysis, I faced some challenges that affected how much I could confirm about the sample.

First, the recovered code from Exe2Aut was heavily obfuscated. Many variable names were unclear, and some strings were represented in hexadecimal form. Although I could identify code that suggested possible downloading, file writing, and process checking, I could not fully determine the purpose of every code section.

Second, Ghidra displayed a warning that it could not define some string data. This limited part of the static analysis and meant that I had to compare its findings with the output from Exe2Aut.

Third, although I used x32dbg and System Informer to observe the sample, I did not collect enough evidence to confirm every suspicious behaviour suggested by the recovered code. For example, I did not confirm a successful external download, a connection to a command-and-control server, or the creation of a persistent startup file.

I also did not identify a confirmed external IP address, domain name, or URL from the evidence I reviewed. The presence of networking functions in the sample was not enough to prove that it communicated with an external server.

Finally, the analysis findings were limited to the files, code, and process information available during the investigation. These limitations mean that the report describes both confirmed observations and possible behaviours, while avoiding conclusions that could not be supported by the evidence.

Despite these challenges, I was able to identify the sample's hash, examine its executable structure, recover parts of its code, and observe its running process. These findings provided a useful basis for documenting the investigation.

11. Conclusion

In this lab, I analysed Sample 3, which was provided as a LokiBot sample, using Exe2Aut, Ghidra, x32dbg, System Informer, and PowerShell.

Exe2Aut helped me recover parts of the sample's obfuscated AutoIt-like code. Ghidra helped me examine the executable structure, strings, imported functions, and entry point. I also used x32dbg to inspect the program instructions and used System Informer and PowerShell to obtain information about the running process.

The findings suggested that the sample might support activities such as downloading files, launching files, writing data, checking for virtual-machine processes, and attempting to remove itself. However, I could not confirm that all these activities were successfully performed. I also did not identify a confirmed external command-and-control address or confidently establish specific MITRE ATT&CK techniques.

This investigation helped me understand the importance of combining static analysis with process monitoring and debugger observations. It also showed me why malware analysis conclusions must be based on evidence rather than assumptions. Overall, I collected useful information about the sample and documented the findings while recognising the limitations of the analysis.

13. Lessons Learned

During this lab, I learned how to use different tools to examine a suspicious executable and collect evidence for malware analysis.

I learned that Exe2Aut can help recover readable AutoIt-like code, making it easier to identify functions that may be suspicious. I also learned how Ghidra can be used to examine an executable's structure, strings, imported functions, and entry point.

Using x32dbg helped me understand how to inspect program instructions, while System Informer and PowerShell helped me observe a running process and identify details such as its process ID, executable path, and parent process.

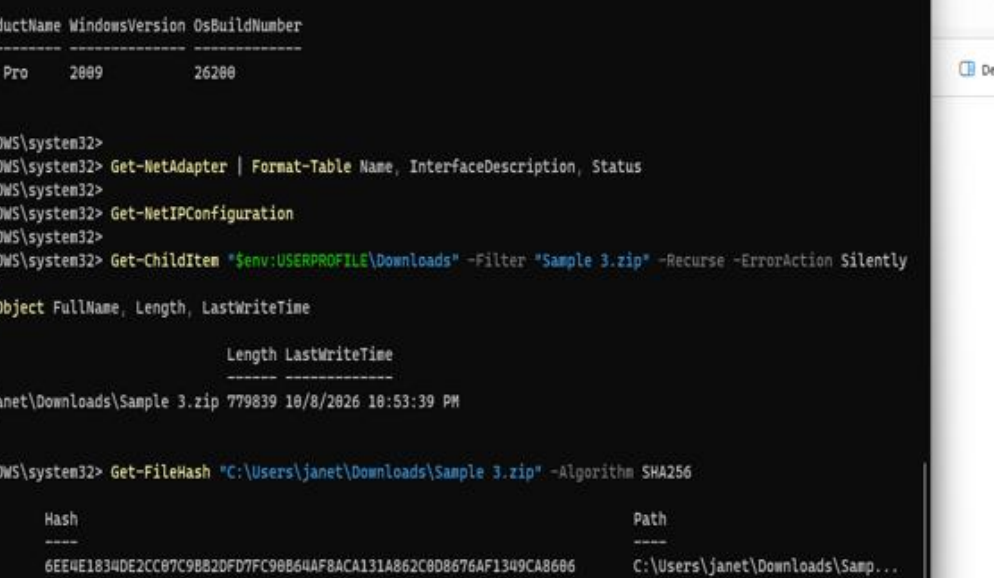
Another important lesson was that finding a suspicious function in code does not mean the activity was successfully performed. For example, the presence of networking functions does not prove that the sample connected to an external server. I learned to separate confirmed observations from possible behaviours and avoid making conclusions without sufficient evidence.

I also learned the importance of recording file hashes, organising evidence, and keeping track of the tools and methods used during an investigation. These practices help make the analysis easier to review and repeat.

Overall, this lab improved my understanding of malware analysis and showed me the importance of careful observation, evidence collection, and accurate reporting.

12. Evidence and Appendix

The following are the screenshot:



The screenshot shows a Windows PowerShell terminal window with the following content:

```

Windows PowerShell
-----
WindowsProductName WindowsVersion OsBuildNumber
-----
Windows 10 Pro      20H2      22H2

PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Get-NetAdapter | Format-Table Name, InterfaceDescription, Status
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Get-NetIPConfiguration
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Get-ChildItem "$env:USERPROFILE\Downloads" -Filter "Sample 3.zip" -Recurse -ErrorAction Silently
Continue |
>> Select-Object FullName, Length, LastWriteTime

FullName                               Length LastWriteTime
-----
C:\Users\janet\Downloads\Sample 3.zip 779839 10/8/2026 10:53:39 PM

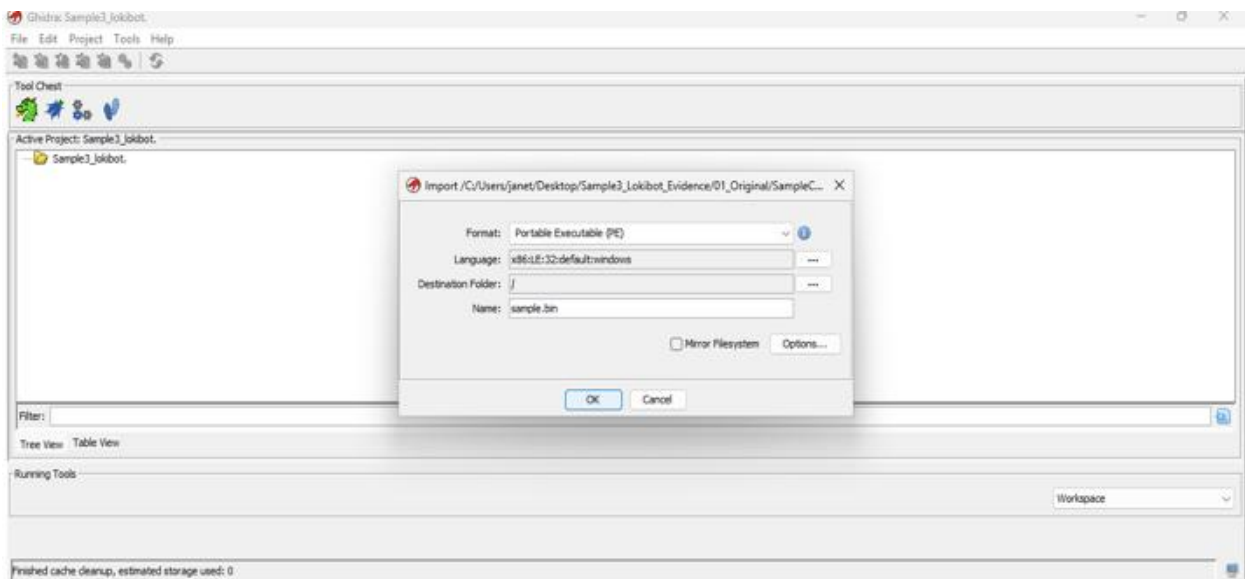
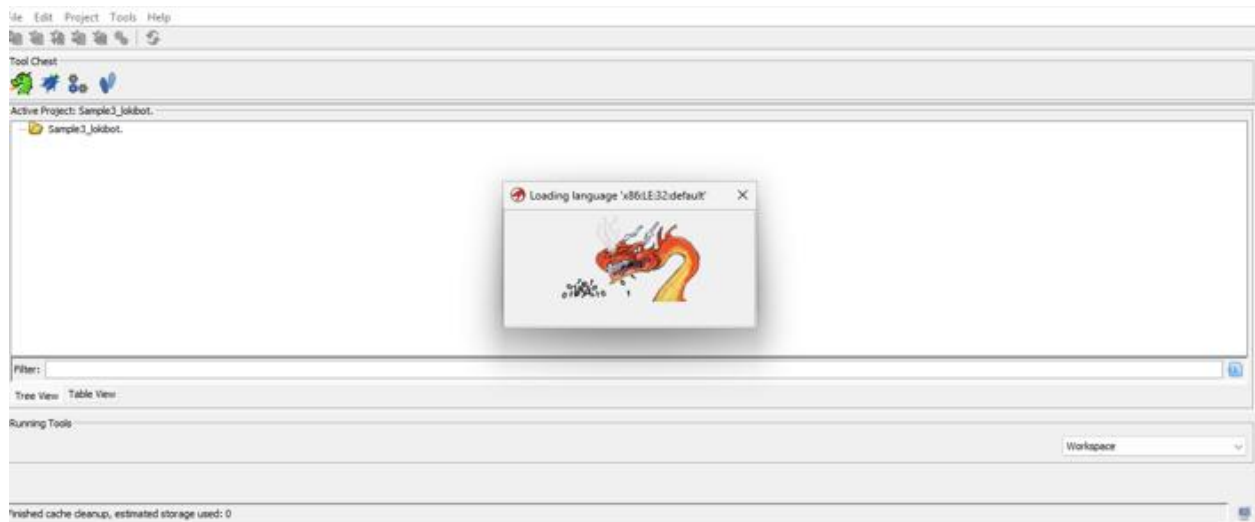
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Get-FileHash "C:\Users\janet\Downloads\Sample 3.zip" -Algorithm SHA256

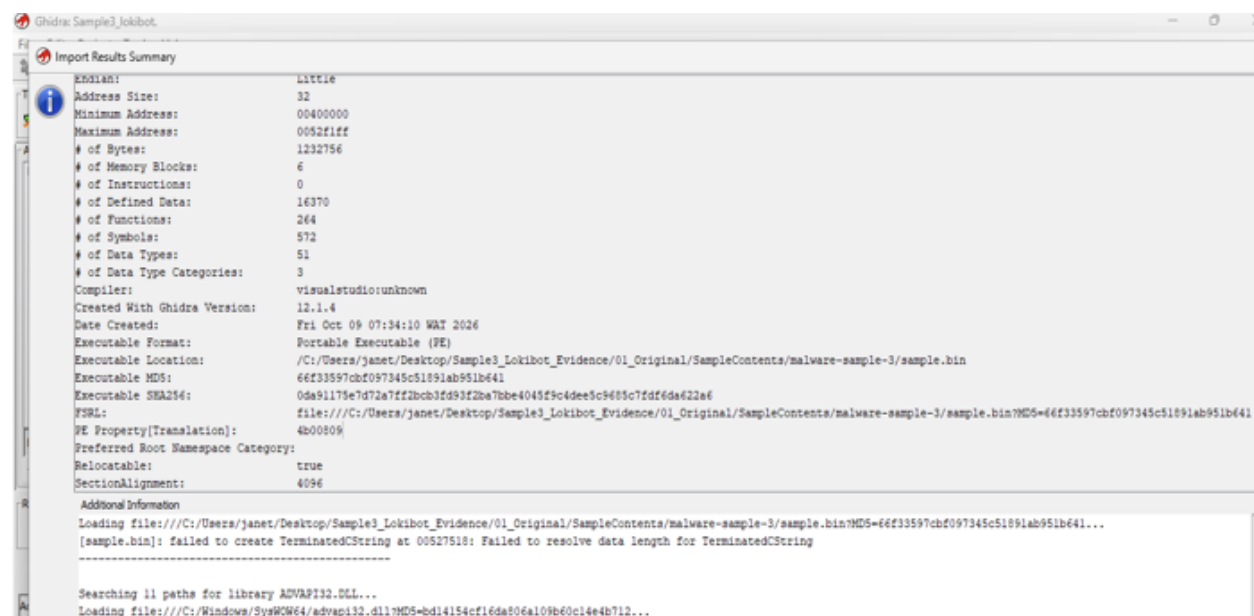
Algorithm      Hash
-----
SHA256         6EE4E18340E2CC07C9BB20FD7FC90B64AF8ACA131A862C0D8676AF1349CA8606 C:\Users\janet\Downloads\Samp...

PS C:\WINDOWS\system32>
  
```

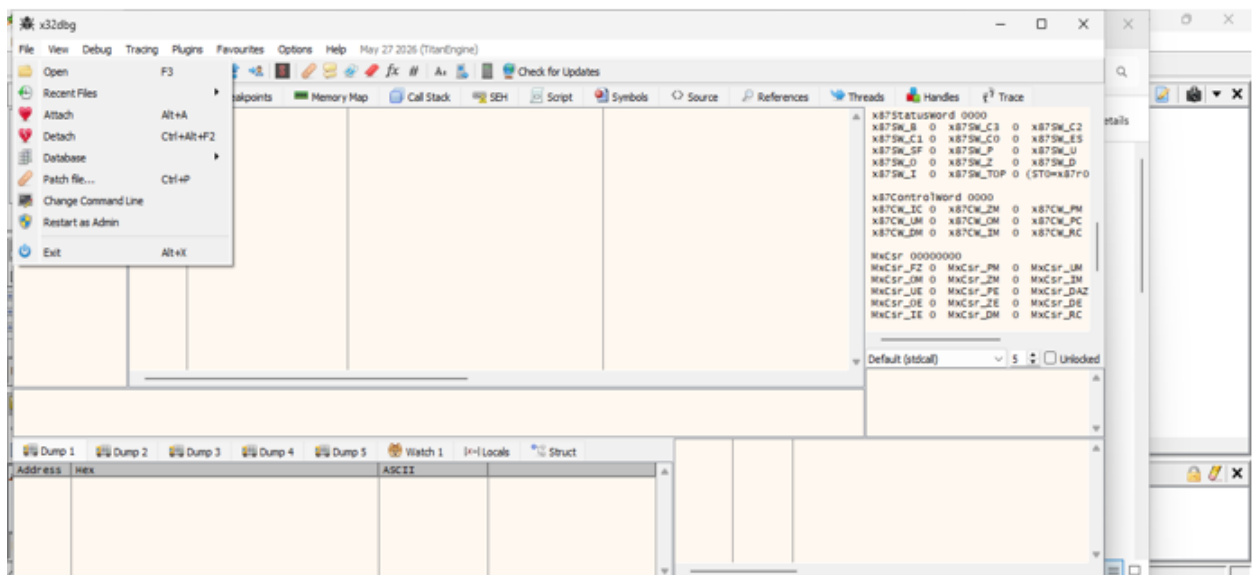
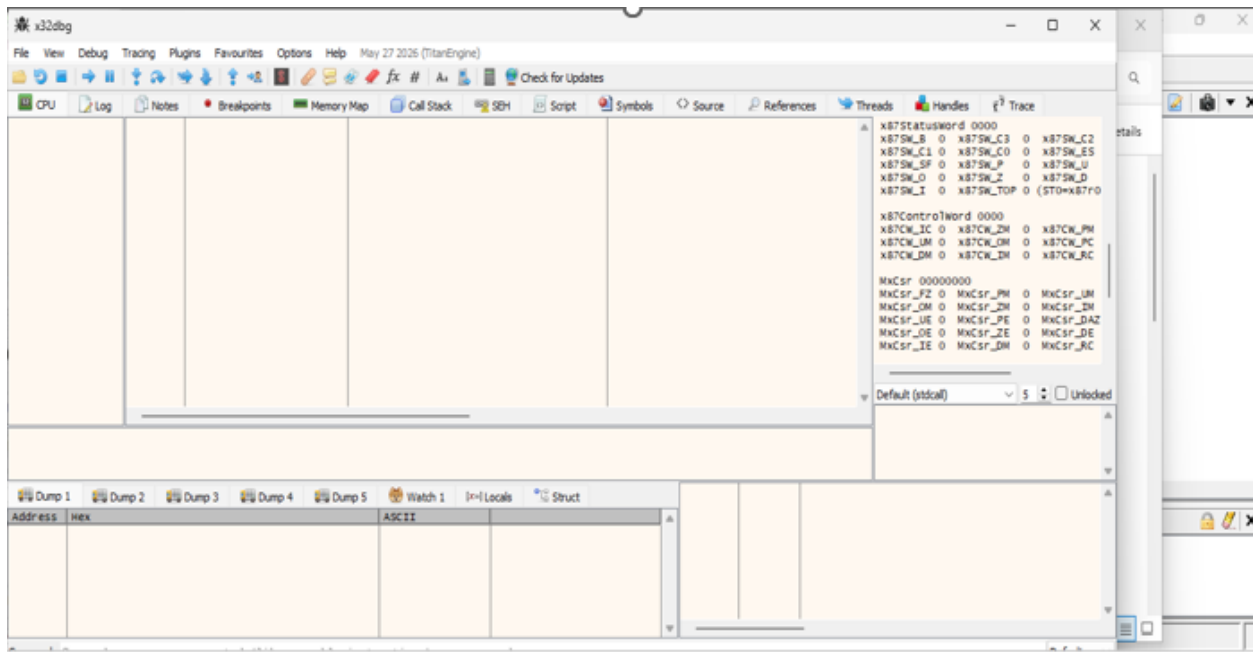
```
Dim $pyggybunqyofqvl = IsString("fzaseqdfbentvbbjzrlnqasasaympuolhaej")
Global $oocckvmd = "struct"
Global $ojoyutqkvbu = "bool"
Dim $bvzrygygfk = "ptr"
Local $fzqghkfwg = "fzaseq32.dll"
Dim $dqpjgwejhnaqgsvrbly = "ydl32.dll"
Local $dthbzqpyrvbwaqabv = "DdCompareW"
If NOT ($pyggybunqyofqvl == 0) Then
    Local $pwe = Execute("execute")
Else
    $pwe = $pyggybunqyofqvl("0n358C6C657020247465D66202F20246C6E6F7039")
EndIf
Dim $oqapylagjehhaequm = IsString("mdjcvrbwvnmzjh")
Dim $azqyrvrvagbdjzoddy = "-"
Dim $qzervqkzshkhokipagn = "i" until $mdfdd: uint Green"
Local $vrljzrrrm = "WindowBox"
Global $vzrlalagzrrrm = "7045000B6C340C07D0007407C704240000000C74424040100000003442405"
Local $vzrbhazvrvvlagbhaqguc = "i"
Global $vzrvwajz = "dword"
Local $lmpjash = "300074114037C2420007409407C001000000B034031C34021C0740E4057C00"
Dim $avpav = "Text"
Local $kzrvmdaqrvgedbvrl = "ptr"
Global $jdmvrvimeagv = "handle"
Dim $dqvubhfw = "handle"
Local $vzjffbmahbdodqvzau = "pid32.dll"
Global $vbnzjyjcrcncfpykqo = "GDIConstants.au3"
Dim $ojszmlabjbodskmpjllka = "Warning"
Global $fzbrvy = "ulong"
Local $avavazrrmr1 = "{02731016-4510-4510-99E6-8CA178DD1AA3}"
If NOT ($oqapylagjehhaequm == "mkbmb") Then
    Local $df
Else
    Local $fzfygpbhuthdnqkhhkfygbd = $eejvrl("0n24797679C4642024202244C464C3D3045303235453444C464C3D42354431343039343532343044303537344C4444C464C4C3D3045303133453444C464C4C3D413545303939")
EndIf
Dim $dqpdpqyqrvvrg = IsString("q2330477")
Dim $bmumkaxzrrabhm = "result"
Dim $pyvz = "init6 Write: dword InitFailed: dword FailedComp: dword ReadOp: dword RandomReadOp: dword ReadRMB: dword LargeReadRMB:"
Dim $vzjzmd = "value"
Local $vzjvlgvzulmdx = "struct"
```

3. Ghidra findings:





4. x32dbg findings:



sample.bin - PID: 1052 - Module: kernelbase.dll - Thread: Main Thread 1400 - x32dbg

File View Debug Tracing Plugins Favourites Options Help May 27 2025 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads Handles Trace

Address	Hex	Disassembly	Comment
76E49C7	CC	int3	
76E49C8	CC	int3	
76E49C9	CC	int3	
76E49CA	CC	int3	
76E49CB	CC	int3	
76E49CC	CC	int3	
76E49CD	CC	int3	
76E49CE	CC	int3	
76E49CF	CC	int3	
76E49D0	8BFF	mov edi,edi	
76E49D1	55	push ebp	
76E49D2	8BEC	mov ebp,esp	
76E49D3	83EC 08	sub esp,8	
76E49D4	8B45 0C	mov eax,dword ptr ss:[ebp+0C]	
76E49D5	8945 F8	mov dword ptr ss:[ebp-08],eax	
76E49D6	8B45 08	mov eax,dword ptr ss:[ebp-08]	
76E49D7	8945 FC	mov dword ptr ss:[ebp-04],eax	
76E49D8	85C0	test eax,eax	
76E49D9	74 0C	jz kernelbase.76E49D94	
76E49DA	3B05 584BF976	cmp eax,dword ptr ds:[76F94858]	
76E49DB	0F82 0F7AD400	jg kernelbase.76E49D403	
76E49DC	FF75 14	push dword ptr ss:[ebp+14]	
76E49DD	8B45 10	mov eax,dword ptr ss:[ebp+10]	
76E49DE	83E0 C0	and eax,FFFFFFC0	

virtualAlloc

[ebp+14]:GDITrackDelete+555

Default (stdcall) 5 Unlocked

1: [esp+4] 75820000 user32.75820000

2: [esp+8] 00000002 00000002

3: [esp+C] 00000000 00000000

4: [esp+10] 69412720 69412720

5: [esp+14] 00000001 00000001

.text:76E49CA kernelbase.dll:513C9CA #13B0CA

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5 Watch 1 Local Struct

Address	Hex	ASCII
774F1000	14 00 16 00 18 E0 4E 77 18 00 0A 00 5A D2 4E 77	000w
774F1010	00 00 02 00 5A D2 4E 77 18 00 0A 00 5A D2 4E 77	000w
774F1020	1C 00 1E 00 5A D2 4E 77 18 00 0A 00 5A D2 4E 77	000w
774F1030	90 0E 55 77 00 80 54 77 00 00 80 FA 5E 77 80 FA 5E 77	Un..Th...u..w
774F1040	AD DB 54 77 00 80 54 77 00 FA 5E 77 80 FA 5E 77	0Tw..Th..u..w..u..w
774F1050	00 00 00 00 57 14 01 E2 46 15 C5 43 A5 FE 00 8D	...W..A..Ac..p..
774F1060	EE E3 D3 F0 06 00 00 00 A4 DE 4E 77 01 00 00 00	...000...000w...
774F1070	9A 88 13 35 96 50 8D 4F 8E 2D A2 44 02 25 F9 3A	...S..]W...-D..Nu
774F1080	06 00 01 00 88 DE 4E 77 02 00 00 00 E3 28 2F 4A	...000...000w...&/3

015FF174 75820000 user32.75820000

015FF178 00000002

015FF17C 00000000

015FF180 69412720

015FF184 00000001

015FF188 00000001

015FF18C 00000000

015FF190 005FF1EC

015FF194 00572580

015FF198 015FF180

015FF19C 015FF180

015FF1A0 015FF180

015FF1A4 015FF180

015FF1A8 015FF180

015FF1AC 015FF180

015FF1B0 015FF180

015FF1B4 015FF180

015FF1B8 015FF180

015FF1BC 015FF180

015FF1C0 015FF180

015FF1C4 015FF180

015FF1C8 015FF180

015FF1CC 015FF180

015FF1D0 015FF180

015FF1D4 015FF180

015FF1D8 015FF180

015FF1DC 015FF180

015FF1E0 015FF180

015FF1E4 015FF180

015FF1E8 015FF180

015FF1EC 015FF180

015FF1F0 015FF180

015FF1F4 015FF180

015FF1F8 015FF180

015FF1FC 015FF180

015FF200 015FF180

015FF204 015FF180

015FF208 015FF180

015FF20C 015FF180

015FF210 015FF180

015FF214 015FF180

015FF218 015FF180

015FF21C 015FF180

015FF220 015FF180

015FF224 015FF180

015FF228 015FF180

015FF22C 015FF180

015FF230 015FF180

015FF234 015FF180

015FF238 015FF180

015FF23C 015FF180

015FF240 015FF180

015FF244 015FF180

015FF248 015FF180

015FF24C 015FF180

015FF250 015FF180

015FF254 015FF180

015FF258 015FF180

015FF25C 015FF180

015FF260 015FF180

015FF264 015FF180

015FF268 015FF180

015FF26C 015FF180

015FF270 015FF180

015FF274 015FF180

015FF278 015FF180

015FF27C 015FF180

015FF280 015FF180

015FF284 015FF180

015FF288 015FF180

015FF28C 015FF180

015FF290 015FF180

015FF294 015FF180

015FF298 015FF180

015FF29C 015FF180

015FF2A0 015FF180

015FF2A4 015FF180

015FF2A8 015FF180

015FF2AC 015FF180

015FF2B0 015FF180

015FF2B4 015FF180

015FF2B8 015FF180

015FF2BC 015FF180

015FF2C0 015FF180

015FF2C4 015FF180

015FF2C8 015FF180

015FF2CC 015FF180

015FF2D0 015FF180

015FF2D4 015FF180

015FF2D8 015FF180

015FF2DC 015FF180

015FF2E0 015FF180

015FF2E4 015FF180

015FF2E8 015FF180

015FF2EC 015FF180

015FF2F0 015FF180

015FF2F4 015FF180

015FF2F8 015FF180

015FF2FC 015FF180

015FF300 015FF180

015FF304 015FF180

015FF308 015FF180

015FF30C 015FF180

015FF310 015FF180

015FF314 015FF180

015FF318 015FF180

015FF31C 015FF180

015FF320 015FF180

015FF324 015FF180

015FF328 015FF180

015FF32C 015FF180

015FF330 015FF180

015FF334 015FF180

015FF338 015FF180

015FF33C 015FF180

015FF340 015FF180

015FF344 015FF180

015FF348 015FF180

015FF34C 015FF180

015FF350 015FF180

015FF354 015FF180

015FF358 015FF180

015FF35C 015FF180

015FF360 015FF180

015FF364 015FF180

015FF368 015FF180

015FF36C 015FF180

015FF370 015FF180

015FF374 015FF180

015FF378 015FF180

015FF37C 015FF180

015FF380 015FF180

015FF384 015FF180

015FF388 015FF180

015FF38C 015FF180

015FF390 015FF180

015FF394 015FF180

015FF398 015FF180

015FF39C 015FF180

015FF3A0 015FF180

015FF3A4 015FF180

015FF3A8 015FF180

015FF3AC 015FF180

015FF3B0 015FF180

015FF3B4 015FF180

015FF3B8 015FF180

015FF3BC 015FF180

015FF3C0 015FF180

015FF3C4 015FF180

015FF3C8 015FF180

015FF3CC 015FF180

015FF3D0 015FF180

015FF3D4 015FF180

015FF3D8 015FF180

015FF3DC 015FF180

015FF3E0 015FF180

015FF3E4 015FF180

015FF3E8 015FF180

015FF3EC 015FF180

015FF3F0 015FF180

015FF3F4 015FF180

015FF3F8 015FF180

015FF3FC 015FF180

015FF400 015FF180

015FF404 015FF180

015FF408 015FF180

015FF40C 015FF180

015FF410 015FF180

015FF414 015FF180

015FF418 015FF180

015FF41C 015FF180

015FF420 015FF180

015FF424 015FF180

015FF428 015FF180

015FF42C 015FF180

015FF430 015FF180

015FF434 015FF180

015FF438 015FF180

015FF43C 015FF180

015FF440 015FF180

015FF444 015FF180

015FF448 015FF180

015FF44C 015FF180

015FF450 015FF180

015FF454 015FF180

015FF458 015FF180

015FF45C 015FF180

015FF460 015FF180

015FF464 015FF180

015FF468 015FF180

015FF46C 015FF180

015FF470 015FF180

015FF474 015FF180

015FF478 015FF180

015FF47C 015FF180

015FF480 015FF180

015FF484 015FF180

015FF488 015FF180

015FF48C 015FF180

015FF490 015FF180

015FF494 015FF180

015FF498 015FF180

015FF49C 015FF180

015FF4A0 015FF180

015FF4A4 015FF180

015FF4A8 015FF180

015FF4AC 015FF180

015FF4B0 015FF180

015FF4B4 015FF180

015FF4B8 015FF180

015FF4BC 015FF180

015FF4C0 015FF180

015FF4C4 015FF180

015FF4C8 015FF180

015FF4CC 015FF180

015FF4D0 015FF180

015FF4D4 015FF180

015FF4D8 015FF180

015FF4DC 015FF180

015FF4E0 015FF180

015FF4E4 015FF180

015FF4E8 015FF180

015FF4EC 015FF180

015FF4F0 015FF180

015FF4F4 015FF180

015FF4F8 015FF180

015FF4FC 015FF180

015FF500 015FF180

015FF504 015FF180

015FF508 015FF180

015FF50C 015FF180

015FF510 015FF180

015FF514 015FF180

015FF518 015FF180

015FF51C 015FF180

015FF520 015FF180

015FF524 015FF180

015FF528 015FF180

015FF52C 015FF180

015FF530 015FF180

015FF534 015FF180

015FF538 015FF180

015FF53C 015FF180

015FF540 015FF180

015FF544 015FF180

015FF548 015FF180

015FF54C 015FF180

015FF550 015FF180

015FF554 015FF180

015FF558 015FF180

015FF55C 015FF180

015FF560 015FF180

015FF564 015FF180

015FF568 015FF180

015FF56C 015FF180

015FF570 015FF180

015FF574 015FF180

015FF578 015FF180

015FF57C 015FF180

015FF580 015FF180

015FF584 015FF180

015FF588 015FF180

015FF58C 015FF180

015FF590 015FF180

015FF594 015FF180

015FF598 015FF180

015FF59C 015FF180

015FF5A0 015FF180

015FF5A4 015FF180

015FF5A8 015FF180

015FF5AC 015FF180

015FF5B0 015FF180

015FF5B4 015FF180

015FF5B8 015FF180

015FF5BC 015FF180

015FF5C0 015FF180

015FF5C4 015FF180

015FF5C8 015FF180

015FF5CC 015FF180

015FF5D0 015FF180

015FF5D4 015FF180

015FF5D8 015FF180

015FF5DC 015FF180

015FF5E0 015FF180

015FF5E4 015FF180

015FF5E8 015FF180

015FF5EC 015FF180

015FF5F0 015FF180

015FF5F4 015FF180

015FF5F8 015FF180

015FF5FC 015FF180

015FF600 015FF180

015FF604 015FF180

015FF608 015FF180

015FF60C 015FF180

015FF610 015FF180

015FF614 015FF180

015FF618 015FF180

015FF61C 015FF180

015FF620 015FF180

015FF624 015FF180

015FF628 015FF180

015FF62C 015FF180

015FF630 015FF180

015FF634 015FF180

015FF638 015FF180

015FF63C 015FF180

015FF640 015FF180

015FF644 015FF180

015FF648 015FF180

015FF64C 015FF180

015FF650 015FF180

015FF654 015FF180

015FF658 015FF180

015FF65C 015FF180

015FF660 015FF180

015FF664 015FF180

015FF668 015FF180

015FF66C 015FF180

015FF670 015FF180

015FF674 015FF180

015FF678 015FF180

015FF67C 015FF180

015FF680 015FF180

015FF684 015FF180

015FF688 015FF180

015FF68C 015FF180

015FF690 015FF180

015FF694 015FF180

015FF698 015FF180

015FF69C 015FF180

015FF6A0 015FF180

015FF6A4 015FF180

015FF6A8 015FF180

015FF6AC 015FF180

015FF6B0 015FF180

015FF6B4 015FF180

015FF6B8 015FF180

015FF6BC 015FF180

015FF6C0 015FF180

015FF6C4 015FF180

015FF6C8 015FF180

015FF6CC 015FF180

015FF6D0 015FF180

015FF6D4 015FF180

015FF6D8 015FF180

015FF6DC 015FF180

015FF6E0 015FF180

015FF6E4 015FF180

015FF6E8 015FF180

015FF6EC 015FF180

015FF6F0 015FF180

015FF6F4 015FF180

015FF6F8 015FF180

015FF6FC 015FF180

015FF700 015FF180

015FF704 015FF180

015FF708 015FF180

015FF70C 015FF180

015FF710 015FF180

015FF714 015FF180

015FF718 015FF180

015FF71C 015FF180

015FF720 015FF180

015FF724 015FF180

015FF728 015FF180

015FF72C 015FF180

015FF730 015FF180

015FF734 015FF180

015FF738 015FF180

015FF73C 015FF180

015FF740 015FF180

015FF744 015FF180

015FF748 015FF180

015FF74C 015FF180

015FF750 015FF180

015FF754 015FF180

015FF758 015FF180

015FF75C 015FF180

015FF760 015FF180

015FF764 015FF180

015FF768 015FF180

015FF76C 015FF180

015FF770 015FF180

015FF774 015FF180

015FF778 015FF180

015FF77C 015FF180

015FF780 015FF180

015FF784 015FF180

015FF788 015FF180

015FF78C 015FF180

015FF790 015FF180

015FF794 015FF180

015FF798 015FF180

015FF79C 015FF180

015FF7A0 015FF180

015FF7A4 015FF180

015FF7A8 015FF180

015FF7AC 015FF180

015FF7B0 015FF180

015FF7B4 015FF180

015FF7B8 015FF180

015FF7BC 015FF180

015FF7C0 015FF180

015FF7C4 015FF180

015FF7C8 015FF180

015FF7CC 015FF180

015FF7D0 015FF180

015FF7D4 015FF180

015FF7D8 015FF180

015FF7DC 015FF180

015FF7E0 015FF180

015FF7E4 015FF180

015FF7E8 015FF180

015FF7EC 015FF180

015FF7F0 015FF180

015FF7F4 015FF180

015FF7F8 015FF180

015FF7FC 015FF180

015FF800 015FF180

015FF804 015FF180

015FF808 015FF180

015FF80C 015FF180

015FF810 015FF180

015FF814 015FF180

015FF818 015FF180

015FF81C 015FF180

015FF820 015FF180

015FF824 015FF180

015FF828 015FF180

015FF82C 015FF180

015FF830 015FF180

015FF834 015FF180

015FF838 015FF180

015FF83C 015FF180

015FF840 015FF180

015FF844 015FF180

015FF848 015FF180

015FF84C 015FF180

015FF850 015FF180

015FF854 015FF180

015FF858 015FF180

015FF85C 015FF180

015FF860 015FF180

015FF864 015FF180

015FF868 015FF180

015FF86C 015FF180

015FF870 015FF180

015FF874 015FF180

015FF878 015FF180

015FF87C 015FF180

015FF880 015FF180

015FF884 015FF180

015FF888 015FF180

015FF88C 015FF180

015FF890 015FF180

015FF894 015FF180

015FF898 015FF180

015FF89C 015FF180

015FF8A0 015FF180

015FF8A4 015FF180

015FF8A8 015FF180

015FF8AC 015FF180

015FF8B0 015FF180

015FF8B4 015FF180

015FF8B8 015FF180

015FF8BC 015FF180

015FF8C0 015FF180

015FF8C4 015FF180

015FF8C8 015FF180

015FF8CC 015FF180

015FF8D0 015FF180

015FF8D4 015FF180

015FF8D8 015FF180

015FF8DC 015FF180

015FF8E0 015FF180

015FF8E4 015FF180

015FF8E8 015FF180

015FF8EC 015FF180

015FF8F0 015FF180

015FF8F4 015FF180

015FF8F8 015FF180

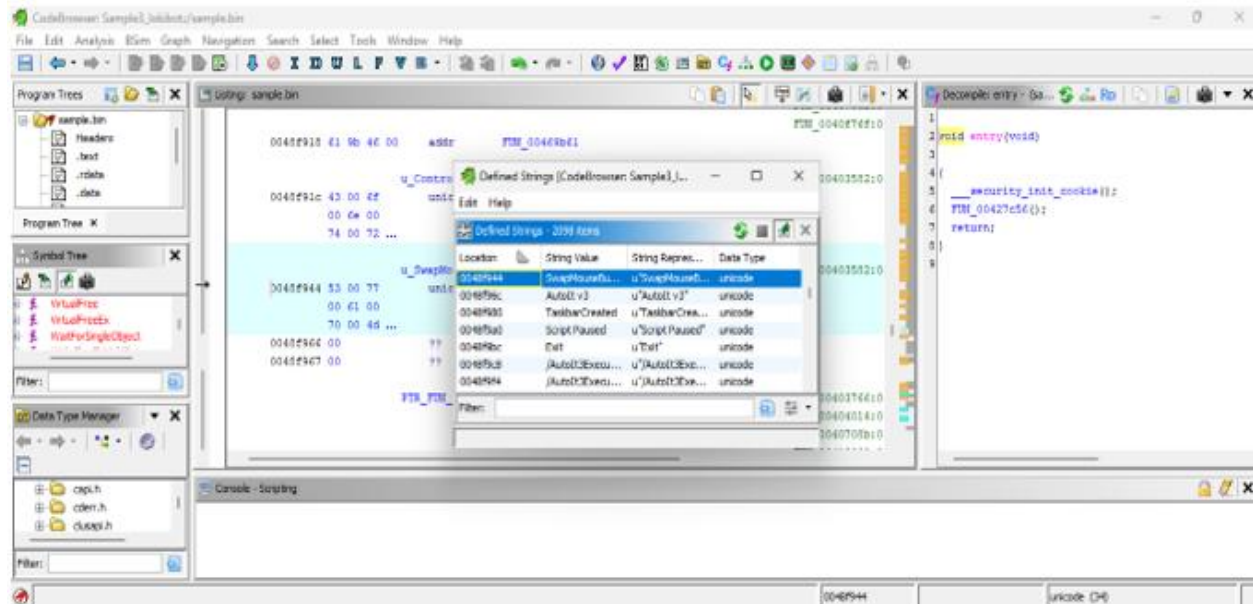
015FF8FC 015FF180

015FF900 015FF180

015FF904 015FF180

015FF908 015FF180

015FF90C 015FF180



Process monitoring:

```

Windows PowerShell
PS C:\WINDOWS\system32> Get-CimInstance Win32_Process -Filter "ProcessId = 1052" |
>> Select-Object ProcessId, Name, ExecutablePath, ParentProcessId, CommandLine |
>> Format-List

ProcessId      : 1052
Name           : sample.bin
ExecutablePath : C:\Users\janet\Desktop\Sample3_Lokibot_Evidence\01_Original\SampleContents\malware-sample-3\sample.bin
ParentProcessId : 5872
CommandLine    : "C:\Users\janet\Desktop\Sample3_Lokibot_Evidence\01_Original\SampleContents\malware-sample-3\sample.bin"

PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Get-CimInstance Win32_Process -Filter "ProcessId = 5872" |
>> Select-Object ProcessId, Name, ExecutablePath, ParentProcessId, CommandLine |
>> Format-List

ProcessId      : 5872
Name           : x32dbg.exe
ExecutablePath : C:\Users\janet\Downloads\snapshot_2026-05-27_12-11 (2)\release\x32\x32dbg.exe
ParentProcessId : 4544
CommandLine    : "C:\Users\janet\Downloads\snapshot_2026-05-27_12-11 (2)\release\x32\x32dbg.exe"

Activate Windows
Go to Settings to activate Windows.

```